

TRAVEL RENTALS

An Integrated Travel Booking Platform

*A Detailed Academic Design and Project Development Report
submitted for the evaluation of Baccalaureate Degree requirements*

**Developed and Compiled by:
Project Development Team
Lokesh Nargesh and Co.**

Department of Computer Science and Engineering
College of Engineering and Technology
November 2025

Certificate of Originality

This is to certify that the project report entitled 'Travel Rentals: Integrated Travel Booking Platform' submitted by the Project Development Team is a record of original work carried out under guidance. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Acknowledgments

We express our sincere gratitude to our project coordinators and advisors for their insightful guidance and support throughout the design and construction of this travel booking platform. We also thank the open-source community for providing modern standards in semantic web models, and our peer reviewers whose feedback has greatly influenced our user experience optimizations.

Table of Contents

1. Introduction.....	4
1.1 Project Overview.....	4
1.2 Scope and Objectives.....	5
2. Literature Survey.....	7
2.1 Evolution of Online Travel Agencies (OTAs).....	7
2.2 Analysis of Segmented Competitor Systems.....	8
3. System Analysis and Requirements (SRS).....	10
3.1 Functional Requirements (FR).....	10
3.2 Non-Functional Requirements (NFR).....	12
4. System Architecture and Design.....	14
4.1 Logical Architecture Tier.....	14
4.2 Technology Stack Deep-Dive.....	15
5. Database Design and Data Modeling.....	17
5.1 JSON Document Entities.....	17
5.2 Document Schemas (JSON).....	18
6. UML and Flow Diagrams.....	20
6.1 Use Case Specifications.....	20
6.2 Data Flow Diagrams (DFD Level 0 & 1).....	21
6.3 Checkout Sequence Interactions.....	23
7. Implementation Methodology.....	25
7.1 Scrum Work Cycles and Milestones.....	25
7.2 Codebase Structure Layout.....	26
8. Quality Assurance and Testing.....	27
8.1 Unit and Integration Testing.....	27
8.2 User Acceptance Testing (UAT).....	28
9. Conclusion and Future Roadmap.....	30

Chapter 1: Introduction

1.1 Project Overview

The Travel Rentals platform has been developed as an integrated technical solution to address the increasing fragmentation in the modern online travel booking market. Over the last two decades, digital travel coordination has transitioned from a manual travel agency consultation workflow into a self-service browser workflow. While this transition has granted users more options, it has introduced significant operational inefficiencies. Today, planning a basic holiday or corporate trip requires individuals to interact with multiple independent vertical booking engines.

The primary focus of this project is to consolidate these independent vertical processes - namely flight reservations, hotel room bookings, and rental car reservations - into a single web application. By offering a unified interface, the platform mitigates the cognitive friction and redundancy associated with modern travel planning. Rather than requiring users to open multiple browser tabs and enter redundant details on separate systems, Travel Rentals coordinates these steps into an integrated cart-based checkout workflow. This ensures that travel segments are chronologically aligned and price-optimized.

1.1.1 Problem Statement

The current state of digital travel coordination places an unnecessary cognitive load on the consumer. Users wishing to plan a complete itinerary must search, compare, and book through multiple vendors. This disjointed process results in several key issues:

- * **High Cognitive Load:** Consumers must manage multiple user accounts, distinct navigation flows, and differing pricing mechanisms across multiple aggregator websites simultaneously.

- * **Data Redundancy:** Crucial user data (including contact info, passenger data, billing addresses, and credit card details) must be entered repeatedly on each platform.

- * Itinerary Discrepancies: Since booking engines do not coordinate with one another, travelers face a high risk of timing conflicts, such as selecting a flight that arrives after the car rental location has closed.

1.2 Scope and Objectives

The scope of this project is to build a functional Minimum Viable Product (MVP) that integrates the three primary travel verticals - Flights, Hotels, and Cars. Ancillary services, such as travel insurance, local tour activities, and public transit routing, are excluded from this phase. The project objectives include:

- * Development of a premium, responsive front-end interface styled with modern glassmorphism principles.

- * Implementation of a centralized database layer mock simulator using local web storage to store user sessions and cart items.

- * Integration of an interactive search, filter, and sort engine across all travel segments.

- * Simulation of secure payment transactions with tokenized card storage to prevent storing raw card numbers.

Chapter 2: Literature Survey

2.1 Evolution of Online Travel Agencies (OTAs)

Online Travel Agencies (OTAs) emerged in the mid-1990s as front-ends for Global Distribution Systems (GDS) such as Sabre and Amadeus. Early platforms like Expedia and Travelocity allowed consumers to bypass physical travel agents. However, these early systems were built on legacy mainframe interfaces, leading to slow response times and rigid search options. Over time, web service APIs enabled these systems to aggregate real-time data from hundreds of airlines and hotel chains, resulting in the modern aggregator market.

Despite these technical advances, modern OTAs remain highly segmented. While companies like Booking Holdings and Expedia Group own various brands, their underlying search engines operate as separate entities. This fragmentation is primarily driven by distinct vertical inventories: flight search requires querying dynamic airline seat allocation systems, hotel booking demands real-time room availability checks, and car rentals rely on local fleet distribution schedules. This division has prevented the industry from building a truly unified, single-session booking pipeline.

2.2 Analysis of Segmented Competitor Systems

To design an optimal solution, we analyzed three distinct travel booking models currently operating in the market:

- * Skyscanner (Flight Meta-Search): Excellent for dynamically comparing ticket prices across multiple airlines. However, it operates purely as a redirect portal, meaning it does not handle hotel stays or car rentals, forcing users to leave the platform to complete their bookings.

- * Booking.com (Accommodation Specialist): A market leader in hotel and vacation home listings. While it has recently added flight and car search through third-party integrations,

these features function as separate modules with no unified cart, preventing users from checking out with a single payment.

* Hertz / Enterprise (Car Rental Verticals): Proprietary systems focusing solely on fleet management. These systems offer no integration with flight arrivals, leaving users to manually coordinate pick-up times with flight delays.

This comparative analysis highlights the clear market need for a unified platform like Travel Rentals. By providing a single checkout session for all three verticals, the platform simplifies the user journey, coordinates arrival and pick-up times, and eliminates redundant data entry.

Chapter 3: System Analysis and Requirements (SRS)

3.1 Functional Requirements (FR)

Functional requirements define the core capabilities and services that the Travel Rentals system must provide to its users. These are categorized into five primary modules:

3.1.1 FR 1: User Management and Authentication

- * Registration: Users must be able to create a secure account by submitting their name, email, and password.
- * Session Control: The system must maintain login states across page refreshes, automatically updating the navigation bar to reflect the user's status.
- * Tokenized Cards: Users must be able to add credit cards, which the system tokenizes for secure, rapid checkout in future sessions.

3.1.2 FR 2: Car Rental Module

- * Search criteria: Users can query cars by city (Delhi, Mumbai, Goa, etc.), pickup date, drop-off date, and pickup time.
- * Refinement: Users can filter vehicles by class (SUV, Sedan, Hatchback), transmission (Automatic, Manual), and fuel (Petrol, Diesel, Electric).
- * Price Tallying: Automatically calculates base rates multiplied by days and provides a cost breakdown on the card.

3.1.3 FR 3: Hotel Booking Module

- * Search criteria: Users can search hotels by destination, check-in, check-out, and guests count.
- * Verification: Displays star ratings, guest scores, and amenity icons (Pool, Gym, Spa).

- * Room Select: Allows users to view and select room types (Standard, Deluxe, Suite) within the hotel card.

3.1.4 FR 4: Flight Booking Module

- * Search parameters: Queries flights by origin, destination, departure date, and seat class.
- * Price Tracking: Users can toggle a price tracker, which monitors the route and alerts them to fluctuations.

3.1.5 FR 5: Unified Checkout and Voucher Generation

- * Cart Management: Integrates flight, hotel, and car items into a unified cart, calculating tax totals automatically.
- * Invoice Export: Generates a printable, styled voucher for confirmed bookings.

3.2 Non-Functional Requirements (NFR)

Non-functional requirements specify the operational criteria and constraints of the Travel Rentals platform:

- * Performance: All filters and sorting actions must update the DOM instantly (under 200ms) to ensure a smooth, lag-free user experience.
- * Security: Payment details must be tokenized, and user accounts stored in sandboxed local storage, protecting user data from unauthorized access.
- * Usability: The platform must feature a premium, mobile-responsive layout styled with glassmorphism principles, including a print-friendly CSS sheet for vouchers.

Chapter 4: System Architecture and Design

4.1 Logical Architecture Tier

The Travel Rentals platform is designed around a three-tier model, separating the user interface, business logic, and database persistence layers. This modular design simplifies system maintenance, testing, and scaling.

1. **Presentation Tier (UI):** The client-facing layer, built with HTML5 semantic elements and styled with custom CSS3 custom variables. This layer handles user interactions, filters, layouts, and print-formatting.

2. **Application Logic Tier:** Built on modular JavaScript scripts (db.js, auth.js, app.js). This tier processes searches, coordinates dates, manages active session data, and simulates secure payment tokens.

3. **Database persistence Tier:** Consists of a structured, local database using HTML5 local storage. This layer stores user accounts, payment tokens, price alert preferences, and booking records, preserving data across sessions.

4.2 Technology Stack Deep-Dive

To ensure maximum performance and maintain compatibility with existing pages, the platform utilizes vanilla web technologies:

- * **Semantic HTML5:** Provides structure, page layout metadata, and accessibility controls.
- * **Custom CSS3:** Implements a variables-based design system, support grids, glassmorphic cards, and media print stylesheets.
- * **Vanilla ES6+ JS:** Handles dynamic UI updates, manages session state, and implements database controls without heavy node compilation pipelines.

* Web Storage API: Uses sandboxed localStorage databases to persist data across page views.

Chapter 5: Database Design and Data Modeling

5.1 JSON Document Entities

Given the NoSQL, document-based architecture of LocalStorage, data modeling is structured around JSON objects. The database is organized into four main tables:

- * tr_users: Stores user records, containing user credentials, email addresses, names, joining dates, and their tokenized credit card details.
- * tr_bookings: Stores transactions containing unique IDs, totals, status flags, and embedded reservation data for flights, hotels, or rental cars.
- * tr_cart: A temporary table storing the user's active selections before checkout.
- * tr_price_alerts: Stores routes monitored by the user for simulated price changes.

5.2 Document Schemas (JSON)

5.2.1 User Record Table Structure

```
{
  "id": "usr_1715694212",
  "name": "Lokesh Nargesh",
  "email": "lokesh@example.com",
  "passwordHash": "pass123",
  "paymentCards": [
    {
      "id": "card_1715694300",
      "cardHolder": "LOKESH NARGESH",
      "last4": "5555",
      "expiry": "11/29",
      "cardType": "Visa",
      "token": "tok_visa_k8a2jd"
    }
  ]
}
```

5.2.2 Booking Record Table Structure

```
{
  "bookingId": "TR-JK82D9S3A",
  "userId": "usr_1715694212",
  "date": "04/06/2026",
  "status": "Confirmed",
  "totalPrice": 16400,
  "flight": {
    "airline": "Vistara",
    "code": "UK-102",
    "from": "DEL",
    "to": "BOM",
    "price": 5000
  },
  "hotel": {
    "name": "The Grand Plaza",
    "roomType": "Standard Room",
    "nights": 2,
    "price": 9000
  },
  "car": null
}
```

Chapter 6: UML and Flow Diagrams

6.1 Use Case Specifications

The use case specification defines the primary interactions supported by the system:

- * User Login/Register: Creates a new user profile or verifies credentials against local storage.
- * Search Travel Options: Allows users to query the mock database for flights, hotels, or rental cars.
- * Manage Cart: Supports adding or removing travel segments before final checkout.
- * Settle Payment: Processes payments using saved token cards or a new card.
- * Generate Voucher: Renders and formats a printable travel voucher for confirmed bookings.

6.2 Data Flow Diagrams (DFD Level 0 & 1)

The DFD Level 1 diagram outlines the flow of data between the user, the core processes, and the database stores:

- * Process 1.0 (Search and Filter): Processes search queries, reads from the inventory database, and returns filtered results to the user interface.
- * Process 2.0 (User Auth): Validates credentials, saves session variables, and manages tokenized payment cards.
- * Process 3.0 (Cart Orchestration): Manages cart updates, calculates date differences, and sums item prices before checkout.
- * Process 4.0 (Transaction Processing): Computes taxes, verifies card tokens, clears the cart, and writes booking records to the database.

6.3 Checkout Sequence Interactions

The sequence diagram illustrates the step-by-step interactions between the presentation layer, session controllers, database modules, and local storage during checkout:

- * 1. The user clicks 'Pay & Confirm' on the checkout page.
- * 2. The page calls the session controller to verify the user state and card token.
- * 3. The controller verifies the session and returns the verified card token.
- * 4. The checkout page sends the booking payload to the database module.
- * 5. The database module generates a unique booking ID, writes the record to local storage, and clears the cart.
- * 6. The database module returns a success confirmation, and the page displays the success modal and receipt.

Chapter 7: Implementation Methodology

7.1 Scrum Work Cycles and Milestones

The project was developed using Agile Scrum principles, organized into four key milestones:

- * Milestone 1 (Design Systems): Designed and implemented the global styling system in `css/style.css`, setting up layout grids, variables, and responsive controls.
- * Milestone 2 (Database and Auth): Developed `js/db.js` and `js/auth.js`, building the client-side database modules, session management, and card tokenization.
- * Milestone 3 (Booking Modules): Implemented the search, filter, and booking flows for flights, hotels, and cars across `flights.html`, `hotels.html`, and `cars.html`.
- * Milestone 4 (Checkout and QA): Built the checkout page, user dashboard, voucher printing styles, and conducted comprehensive testing.

7.2 Codebase Structure Layout

The codebase is organized into a clean, modular structure:

- * `index.html`: The platform landing page, featuring quick-search widgets.
- * `flights.html`: Flight search, results listing, sorting, and price tracking.
- * `hotels.html`: Hotel listings, amenities filters, and room selection.
- * `cars.html`: Car search, class filters, base rate calculation, and booking.
- * `checkout.html`: Centralized checkout page, featuring tax calculation and payment card input.
- * `login.html`: Account registration and login forms.
- * `dashboard.html`: User dashboard, profile settings, saved cards, and voucher printing.

- * `css/style.css`: Unified stylesheet with a dedicated print layout stylesheet.
- * `js/db.js`: Mock database tables and local storage read/write controllers.
- * `js/auth.js`: Handles user session validation and card tokenization.
- * `js/app.js`: Handles navbar updates, date differences, and background price alerts.

Chapter 8: Quality Assurance and Testing

8.1 Unit and Integration Testing

Quality assurance was integrated throughout the development process. Unit tests were conducted for core JavaScript functions, such as date validation and price multiplication. Integration testing focused on the communication between the modules - ensuring that adding an item to the cart in flights.html or hotels.html immediately reflected in the navbar count and compiled correctly on the checkout page.

We also verified the LocalStorage read/write operations to ensure that session data and card tokens remained persistent across browser refreshes. Finally, we tested the print layout stylesheets to confirm that printing a voucher automatically hid interface elements like the navbar and buttons, generating a clean, formal document.

8.2 User Acceptance Testing (UAT)

To validate that the system meets real-world needs, we defined and executed a comprehensive User Acceptance Testing (UAT) matrix. This testing confirmed that the unified booking pipeline works seamlessly:

ID	Feature	Test Scenario	Expected Outcome	Status
TC-01	Registration	Submit sign-up credentials	User saved; logged in navbar	Pass
TC-02	Tokenization	Add card in settings	Saves last 4; returns token	Pass
TC-03	Price Tracking	Toggle tracking on route	Toast alerts fire on change	Pass
TC-04	Stay Tally	Select Deluxe room (3 nights)	Multiplies nights by base rate	Pass
TC-05	Checkout	Complete payment checkout	Booking confirmed; cart empty	Pass
TC-06	Voucher Print	Print invoice voucher	Voucher renders without UI	Pass

Chapter 9: Conclusion and Future Roadmap

9.1 Project Conclusion

The Travel Rentals platform successfully addresses the fragmentation challenges in the online travel booking industry. By unifying the flight, hotel, and car rental search and booking flows into a single, intuitive interface, the platform simplifies the user journey, reduces cognitive load, and eliminates redundant data entry.

By utilizing modern client-side storage technologies, the platform operates efficiently without requiring complex server backends. The application of responsive grids and glassmorphism styling variables ensures a premium look and feel, while the print-friendly layouts make it easy to generate formal travel vouchers. This establishes a solid foundation for a production-scale travel agency portal.

9.2 Future Roadmap

9.2.1 Phase 1: Real-time API Integration

The next phase of development focuses on replacing the mock database module (db.js) with real RESTful APIs. This includes connecting to Global Distribution Systems (GDS) like Amadeus or Sabre for real-time flight inventory and seat allocation, and linking to hotel and car rental provider networks.

9.2.2 Phase 2: Mobile Apps and AI Planning

Future iterations also include launching native iOS and Android applications to utilize mobile capabilities like biometric login and location-based car rentals. Additionally, we plan to implement machine learning models to analyze user preferences and booking patterns, providing personalized itinerary suggestions and optimal booking bundle combinations.

References

- [1] World Wide Web Consortium. 'Web Storage API Standards.' (2023).
- [2] Pressman, R. 'Software Engineering: A Practitioner's Approach.' 9th Edition.
- [3] Nielsen, J. 'Heuristic Evaluation of User Interfaces.' ACM (1994).